

A Framework for Probabilistic Oracles in Distributed Systems

A survey

Luca Lombardo

The Impedance Mismatch

Partitioned Distributed Architecture

Data partitioned across nodes for scalability. Each node maintains local state for its partition. Queries for non-local partitions incur network latency: $t_{\text{network}} \gg t_{\text{local}}$.

Problem: Queries may require remote state access. Cross-partition operations dominate cost.

Critical Operations

- **Membership Testing:** Determine $x \in S$ where S spans multiple partitions. Cost: one round-trip if x not local.
- **Cardinality Estimation:** Compute $|S|$ for partitioned dataset. Exact COUNT(DISTINCT) requires shuffle. Cost: $\Theta(n \cdot |x|)$.

Requirements for Compact Summaries

Resolving the impedance mismatch requires compact summaries satisfying:

- 1 **Local Queryability:** Queryable locally or transmitted with fixed cost independent of dataset size
- 2 **State Synchronization:** Synchronize with datasets undergoing insertion *and deletion*
- 3 **Order-Independent Merge:** Support merge operations producing results identical to single-pass processing

Two Properties Addressing These Conditions

Dynamic Mutability → Condition 2

Composability → Condition 3

These properties impose conflicting requirements on state representation.

The Dictionary Problem

Definition (One-Sided Error Membership Oracle)

A data structure $\mathcal{D}$ representing set S is a one-sided error membership oracle with false positive rate ε if:

- For any $x \in S$: $P(\mathcal{D}.\text{QUERY}(x) = \text{true}) = 1$
- For any $y \notin S$: $P(\mathcal{D}.\text{QUERY}(y) = \text{true}) \leq \varepsilon$

Space Complexity: Deterministic solutions (hash tables) require $\Theta(n \cdot |x|)$ space. Probabilistic solutions require $\Theta(n \log(1/\varepsilon))$ space, independent of element size.

Theorem (Bloom Filter Space-Error Relationship)

To achieve false positive rate ε , a Bloom filter requires $m/n = -\frac{\ln \varepsilon}{(\ln 2)^2} \approx 1.44 \log_2(1/\varepsilon)$ bits per element.

Violated Assumptions in Distributed Systems

Standard Bloom filter construction assumes:

- 1 Set S is static after construction
- 2 Cardinality $|S| = n$ is known at initialization

Reality in Disaggregated Systems:

Datasets undergo continuous insertion and deletion across partitions.
Cardinality is neither fixed nor predictable.

State Synchronization Constraint

An oracle must maintain coherence with dataset evolution. For all times $t > t_{\text{delete}}$ where element x is deleted at time t_{delete} :

$$P(\mathcal{D}.\text{QUERY}(x) = \text{true}) \leq \varepsilon$$

Standard Bloom filters cannot satisfy this constraint.

Stale Positives

Element x deleted from dataset. Bloom filter cannot remove x 's contribution. Bits remain set indefinitely.

Consequence: Queries for deleted x return true. System performs wasted I/O to discover absence. Cost scales with deletion rate.

Capacity Exhaustion

Filter parameterized for cardinality n . When $|S| \gg n$, bit array saturates. False positive rate degrades: $\varepsilon \rightarrow 1$.

Resolution requires over-provisioning or reconstruction with service interruption.

Bloom Filter Variants: Addressing Individual Constraints

Definition (Counting Bloom Filter)

Replace bit array with array C of m counters, each of c bits. Insertion increments counters at $h_i(x)$ for $i \in \{1, \dots, k\}$. Deletion decrements counters. Query returns true if all counters positive.

Properties: Enables deletion without false negatives. Space overhead: $\approx 4\times$ standard Bloom filter for $c = 4$ bits per counter.

Definition (Scalable Bloom Filter)

Ordered sequence of Bloom filters $B_1, B_2, \dots, B_s$ with increasing sizes $m_i > m_{i-1}$ and decreasing error rates $\varepsilon_i < \varepsilon_{i-1}$. New filter appended when active filter reaches load threshold.

Properties: Accommodates unbounded growth. Heterogeneous parameters prevent bitwise union: two SBF instances cannot be merged.

Distributed Bloom Patterns

Definition (Parallel Partitioned Bloom Filter)

Sequence of s homogeneous Bloom filters $B_1, \dots, B_s$. Each filter has identical size m and uses same k hash functions. Union of two ParBF instances computed via bitwise OR of corresponding sub-filters.

Properties: Bitwise OR is commutative, associative, idempotent (enables order-independent merge). No deletion support.

Definition (Distributed Bloom Filter Protocol)

Each node pair (n_i, n_j) uses unique hash family $\mathcal{H}_{ij}$ derived from seed $s_{ij} = ID_i \oplus ID_j$. System-wide false positive rate: $\varepsilon_{\text{System}} \leq \varepsilon^{N-1}$ for N nodes.

Properties: Probabilistic convergence through iterative reconciliation. Filters with different seeds cannot be merged via bitwise OR (no composability). No deletion support.

The Gap in Existing Solutions

Observation: No Bloom filter variant satisfies both requirements.

- CBF: deletion support, but $4\times$ space overhead
- SBF: capacity scaling, but no composability
- ParBF: composability, but no deletion support
- DBF: eventual convergence, but no composability or deletion

Definition (Dynamic Mutability)

A probabilistic oracle $\mathcal{D}$ exhibits dynamic mutability if:

- 1 INSERT(x) and DELETE(x) execute in $O(1)$ time
- 2 Space complexity $O(|S_t| \log(1/\epsilon))$ for current cardinality $|S_t|$
- 3 For all $y \in S_t$: $P(\mathcal{D}.\text{QUERY}(y) = \text{true}) = 1$
- 4 DELETE(x) removes x 's without affecting queries for $y \neq x$

Cuckoo Filter: Structure

Definition (Cuckoo Filter)

A Cuckoo Filter consists of m buckets, each containing b entries. Each entry stores a fingerprint $f = \text{FINGERPRINT}(x)$ of f bits derived from element $x \in \mathcal{U}$.

Definition (Partial-Key Cuckoo Hashing)

For element $x \in \mathcal{U}$, let $f = \text{FINGERPRINT}(x)$ denote a compact bit string representation. Two candidate bucket indices are computed as:

$$i_1 = \text{HASH}(x), \quad i_2 = i_1 \oplus \text{HASH}_f(f)$$

where $\oplus$ denotes bitwise XOR.

Key Property: XOR symmetry enables recovery of i_1 given i_2 and f :

$$i_1 = i_2 \oplus \text{HASH}_f(f)$$

Relocation requires only fingerprint f , not original element x .

Cuckoo Filter: Operations and Guarantees

Operations

- **LOOKUP(x):** Compute i_1, i_2 and check if fingerprint f exists in either bucket. Examines at most $2b$ entries.
- **DELETE(x):** Compute candidate buckets i_1, i_2 . If f exists in either bucket, remove one copy. Executes in $O(1)$ time without auxiliary counters.

Theorem (False Positive Bound)

A Cuckoo Filter with b entries per bucket and fingerprint length f bits satisfies:

$$\epsilon \leq \frac{2b}{2^f}$$

Space Complexity: $O(n \cdot f)$ bits for n elements, where fingerprint size $f \geq \lceil \log_2(1/\epsilon) + \log_2(2b) \rceil$ bits.

Definition (Dynamic Cuckoo Filter)

Ordered sequence of Cuckoo Filters $CF_1, CF_2, \dots, CF_k$. Insertions directed to terminal filter. New filter appended when active filter reaches capacity.

Query traverses sequence until fingerprint found. For n items with per-filter capacity C , sequence length $k = \Theta(n/C)$. Query complexity $O(k) = O(n)$.

Definition (Index-Independent Cuckoo Filter)

Buckets maintained on consistent hash ring. Candidate locations determined by hashing element to ring positions. Bucket identifiers depend only on element hash value, not on total system size.

Query complexity $O(1)$ independent of dataset size n and bucket count m .

Co-Partitioned Architecture

Definition (Co-Partitioned Filter)

System of m nodes with consistent hash ring. Node n_i maintains:

- Data items d where $H(d) \in [t_i, t_{i+1})$
- Filter buckets b where $\text{position}(b) \in [t_i, t_{i+1})$

Query for element x routed to node responsible for $H(x)$.

Theorem (Co-Partitioned Query Latency)

Under uniform query distribution:

$$\mathbb{E}[L_q] = \frac{1}{m} t_{local} + \left(1 - \frac{1}{m}\right) t_{network}$$

Filter partitioning aligned with data partitioning enables local execution. Query performance determined by system scale m , not by number of stored elements.

The Cardinality Estimation Problem

Definition (Cardinality Estimation)

Given a multiset S of elements drawn from universe $\mathcal{U}$, determine $|S|$, the number of distinct elements in S .

Naive Implementation in Data-Parallel Framework:

- *Initial stage*: each parallel worker processes partition to produce local set of unique elements.
- *Shuffle stage*: transmit all unique elements from all partitions to reducer nodes for final aggregation and deduplication.

The Bottleneck

Communication cost proportional to number of unique elements and their size. For datasets with high cardinality, network I/O dominates total execution time.

Challenge: Enable cardinality aggregation without shuffle of raw data.

Requirements for Distributed Aggregation

A distributed cardinality estimation oracle must combine partial estimates from different partitions. The combination operation must satisfy three requirements:

Requirements

- **Error Equivalence:** Result has error bounds identical to oracle constructed in single pass over complete dataset.
- **Cardinality Independence:** Execution time proportional to estimate representation size, not to number of elements.
- **Order Independence:** Final global estimate identical regardless of combination sequence. No coordination required.

These requirements specify a class of binary operations over cardinality estimation oracles.

Definition (Composability)

Let $\mathcal{D}_1, \dots, \mathcal{D}_k$ be instances constructed over partitions $S_1, \dots, S_k$. The oracle exhibits composability if there exists binary operation $\cup$ such that:

- Combined oracle $\mathcal{D}_{agg} = \mathcal{D}_1 \cup \dots \cup \mathcal{D}_k$ has expected relative error asymptotically equivalent to single-pass construction over $S = \bigcup_{i=1}^k S_i$
- Operation $\cup$ is commutative and associative
- For fault tolerance, operation must be idempotent

Consequences: Idempotency enables *at-least-once* delivery without *exactly-once* coordination overhead. These properties transform non-additive COUNT(DISTINCT) into decomposable aggregation.

Definition (HyperLogLog Sketch)

Array M of $m = 2^p$ registers initialized to 0. Hash function $h : \mathcal{U} \rightarrow \{0, 1\}^L$ distributes elements uniformly.

Update Rule: For element v , compute $x = h(v)$. First p bits determine register $j \in \{0, \dots, m-1\}$. Remaining bits w compute observable $\rho(w) = 1 + \text{Iz}(w)$. Update:

$$M[j] := \max(M[j], \rho(w))$$

Cardinality Estimator:

$$E = \alpha_m m^2 \left(\sum_{j=0}^{m-1} 2^{-M[j]} \right)^{-1}$$

Relative error $\approx 1.04/\sqrt{m}$. Space: $\Theta(m \log \log n)$ bits.

Union Operation and Algebraic Properties

Definition (HyperLogLog Union)

Let M_1 and M_2 be sketches representing sets S_1 and S_2 . Union computed as:

$$M_{1 \cup 2}[j] := \max(M_1[j], M_2[j]) \quad \forall j$$

Theorem (Union Equivalence)

Sketch $M_{1 \cup 2}$ equivalent to sketch from single-pass processing of $S_1 \cup S_2$.

Theorem (Algebraic Properties)

Union operation is commutative, associative, and idempotent.

Communication cost becomes $\Theta(k s)$ for k worker nodes transmitting sketches of size s , independent of dataset cardinality n .

HyperLogLog++: Engineering Solution

Definition (HyperLogLog++)

Extension of standard HLL addressing cardinality skew and communication overhead at scale through dual representation and empirical corrections.

Dual Representation

Sparse Format: Variable-length encoded sorted list of (index, value) pairs.

Dense Format: Standard fixed-size array of m registers.

Additional Refinements:

- Empirical bias correction table for low-cardinality accuracy
- 64-bit hash function eliminates large-range correction

Sparse format reduces memory footprint by order of magnitude, directly reducing shuffle communication volume.

Architectural Patterns Enabled by Composability

MapReduce: Two-Level Hierarchy

- Map stage: workers generate local HLL sketches per data partition
- Shuffle stage: transmit compact sketches to reducer nodes
- Reduce stage: apply union operation to merge sketches for each key

Correctness guaranteed by commutativity and associativity of union.

Stream Processing: Temporal Hierarchies

Partition unbounded stream into logical windows. Maintain local sketch per window. Cardinality over interval T composed of sub-intervals

$t_1, \dots, t_k$:

$$M_T = \bigcup_{i=1}^k M_{t_i}$$

Cost independent of raw data volume.

Distinct Constraints

Mutability constrains the *update interface*: oracle must track element-level contributions for targeted removal.

Composability constrains the *merge interface*: oracle state must be derivable from commutative operations.

Conflicting Requirements on State Representation:

- Mutability requires retaining information to identify specific element contributions
- Composability requires information loss through commutative operations discarding element identities

Definition (Reversible Element Representation)

An oracle $\mathcal{D}$ maintains a reversible representation of element x if a deterministic procedure identifies a set of memory locations $L_x \subseteq \text{State}(\mathcal{D})$ such that:

- 1 L_x contains all locations modified by the insertion of x .
- 2 The operation $\text{REMOVE}(x)$ modifies only L_x , preserving the query probability for all $y \neq x$:

$$P(\mathcal{D}.\text{QUERY}(y) = \text{true})_{\text{after}} = P(\mathcal{D}.\text{QUERY}(y) = \text{true})_{\text{before}}$$

The Cuckoo Filter satisfies this by storing explicit fingerprints in discrete buckets. The HyperLogLog sketch violates this by merging contributions into maximum registers.

The Incompatibility Theorem

Lemma (State Indistinguishability)

Let $\mathcal{D}$ possess an idempotent merge operation where $s \cup s = s$. The state s_1 resulting from a single insertion of x is syntactically identical to the state s_2 resulting from merging s_1 with itself.

Theorem (Structural Incompatibility)

Let $\mathcal{D}$ be a probabilistic oracle without auxiliary structures. If $\mathcal{D}$ provides an idempotent merge operation, it cannot support an $O(1)$ deletion operation without introducing false negatives, unless it maintains reversible element representations.

This structural impossibility forces systems to specialize: Cuckoo Filters optimize for mutability, while HyperLogLog optimizes for composability.

Structure Selection Criteria

We propose a decision matrix based on data lifecycle and query scope.

	<i>Single-Partition</i>	<i>Multi-Partition</i>
<i>Fully Dynamic</i> (Insert & Delete)	Cuckoo Filter Requires Mutability	Hybrid Approach Requires Separated Oracles
<i>Static / Append-Only</i> (Immutable / TTL)	Bloom Filter Baseline Efficiency	HyperLogLog Requires Composability

Workloads requiring both deletion and aggregation confront the incompatibility theorem, necessitating hybrid deployments with distinct oracles.